



Politécnico
Internacional

ESTRUCTURA DE DATOS



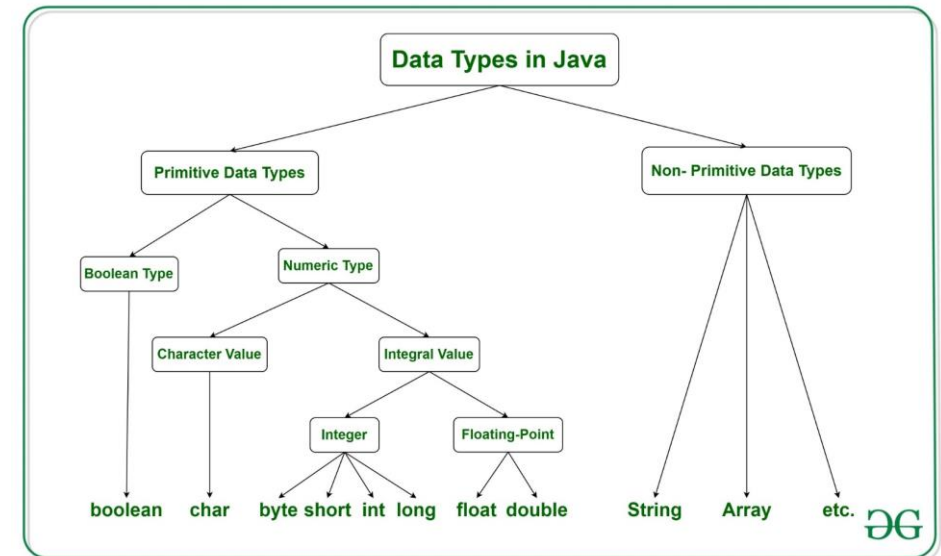
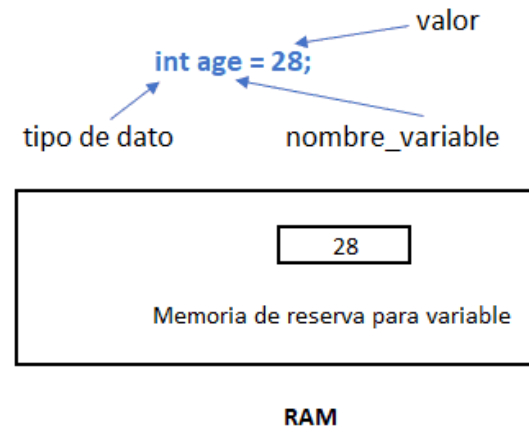
**“Cualquier tecnología suficientemente
avanzada es equivalente a la **magia**.”**

Arthur C. Clarke – Escritor y Científico

AGENDA

- Conceptos Java
- Innova Colombia
- Conclusiones

- ¿Qué es Java?
- Características – Seguridad
- POO
- Sintaxis Básica
- Historia
- Tipado – Estático & Dinámico
- Tipos de datos.
- Operadores
- Variables
- Lectura de Datos
- Estructuras Condicionales





Politécnico
Internacional



COMENTARIOS JAVA



En un programa, los comentarios forman parte en hacer que el programa se vuelva más legible mediante el uso de los detalles del código haciendo que el mantenimiento sea más fácil y pueda encontrarse errores fácilmente. Los comentarios son ignorados por el compilador al compilar un código



TIPOS DE COMENTARIOS



En Java hay tres tipos de comentarios:

- *Comentarios de una sola línea.*
- *Comentarios de múltiples líneas.*
- *Comentarios de la documentación.*

COMENTARIOS 1 LINEA

Sintaxis:

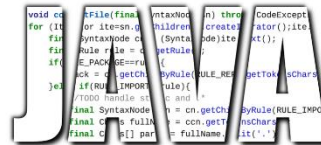
```
//Comentarios aqui ( El texto en esta línea solo se considera comentario )
```

Ejemplo:

```
//Programa Java que muestra comentarios de una sola linea
class Scomment
{
    public static void main(String args[])
    {
        // Comentario de una sola linea aquí
        System.out.println("Comentario de una sola linea arriba");
    }
}
```



Un programador de nivel principiante utiliza comúnmente comentarios de una sola línea para describir la funcionalidad del código. Son los comentarios más fáciles de escribir.





COMENTARIOS VARIAS LINEAS

```
/*El comentario comienza  
continúa  
continúa  
.  
.  
.  
El comentario termina*/
```

Ejemplo:

```
//Programa Java para mostrar comentarios de varias líneas  
class Scomment  
{  
    public static void main(String args[])  
    {  
        System.out.println("Comentarios de varias líneas a continuación");  
        /*Línea de comentario 1  
        Línea de comentario 2  
        Línea de comentario 3*/  
    }  
}
```

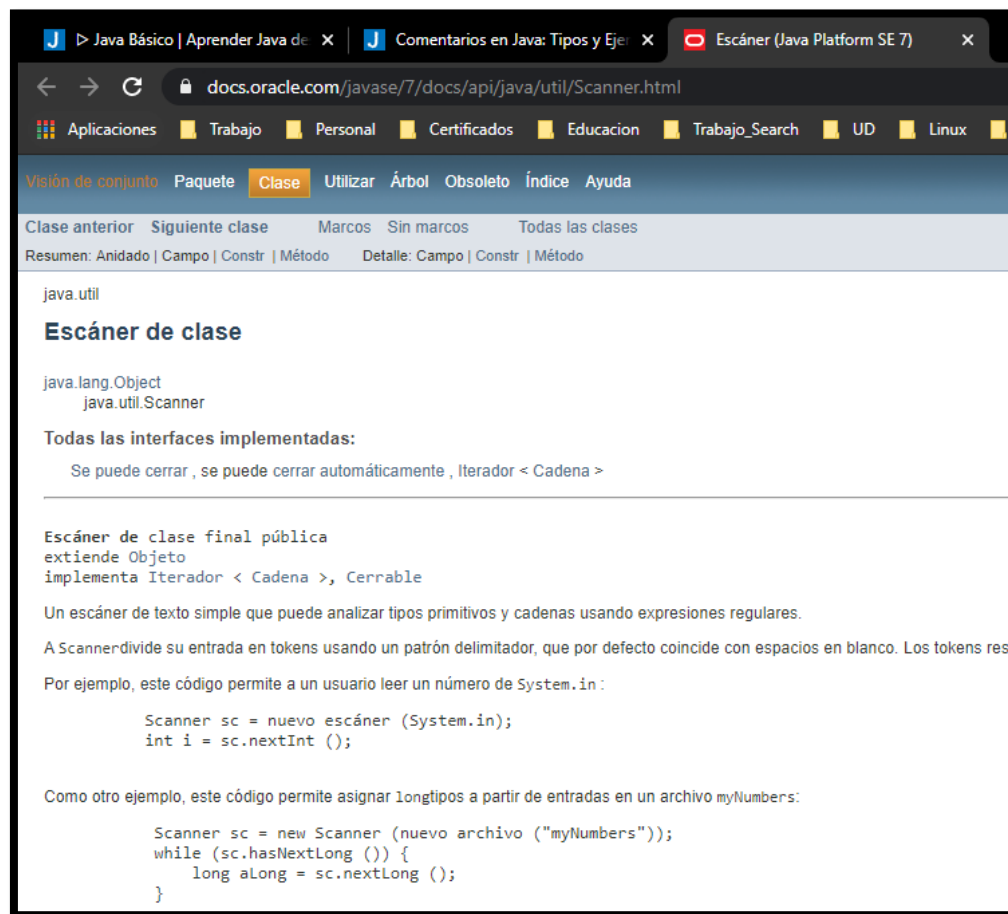


Para describir un método completo en un código o un fragmento complejo, los comentarios de una sola línea pueden ser tediosos de escribir, ya que tenemos que dar ‘//’ en cada línea. Entonces, para evitar repetir ‘//’ en cada línea, podemos utilizar la sintaxis de los comentarios de varias líneas.

COMENTARIOS EN LA DOCUMENTACIÓN



Este tipo de comentarios se utilizan generalmente al escribir código para un paquete de proyecto/software, ya que ayuda a generar una página de documentación para referencia, que puede usarse para obtener información sobre los métodos presentes, sus parámetros, etc.



<https://docs.oracle.com/javase/7/docs/api/java/util/Scanner.html>

COMENTARIOS EN LA DOCUMENTACIÓN

ETIQUETA	DESCRIPCIÓN	SINTAXIS
@author	Agrega el autor de una clase.	@author nombre
{@code}	Muestra texto en la fuente del código sin interpretar el texto como marcado HTML o etiquetas javadoc anidadas.	{@code texto}
{@docRoot}	Representa la ruta relativa al directorio raíz del documento generado desde cualquier página generada.	{@docRoot}
@deprecated	Agrega un comentario que indica que esta API ya no se debe usar.	@deprecated texto-obsoleto
@exception	Agrega un subtítulo "Throws" a la documentación generada, con el nombre de clase y el texto de descripción.	@exception descripción del nombre-clase
{@inheritDoc}	Hereda un comentario de la clase heredable más cercana o la interfaz implementable.	Hereda un comentario de la superclase inmediata.
{@link}	Inserta un enlace con la etiqueta de texto visible que apunta a la documentación del paquete, clase o nombre de miembro especificado de una clase referenciada.	{@linkplain package.class#etiqueta-miembro}

COMENTARIOS EN LA DOCUMENTACIÓN

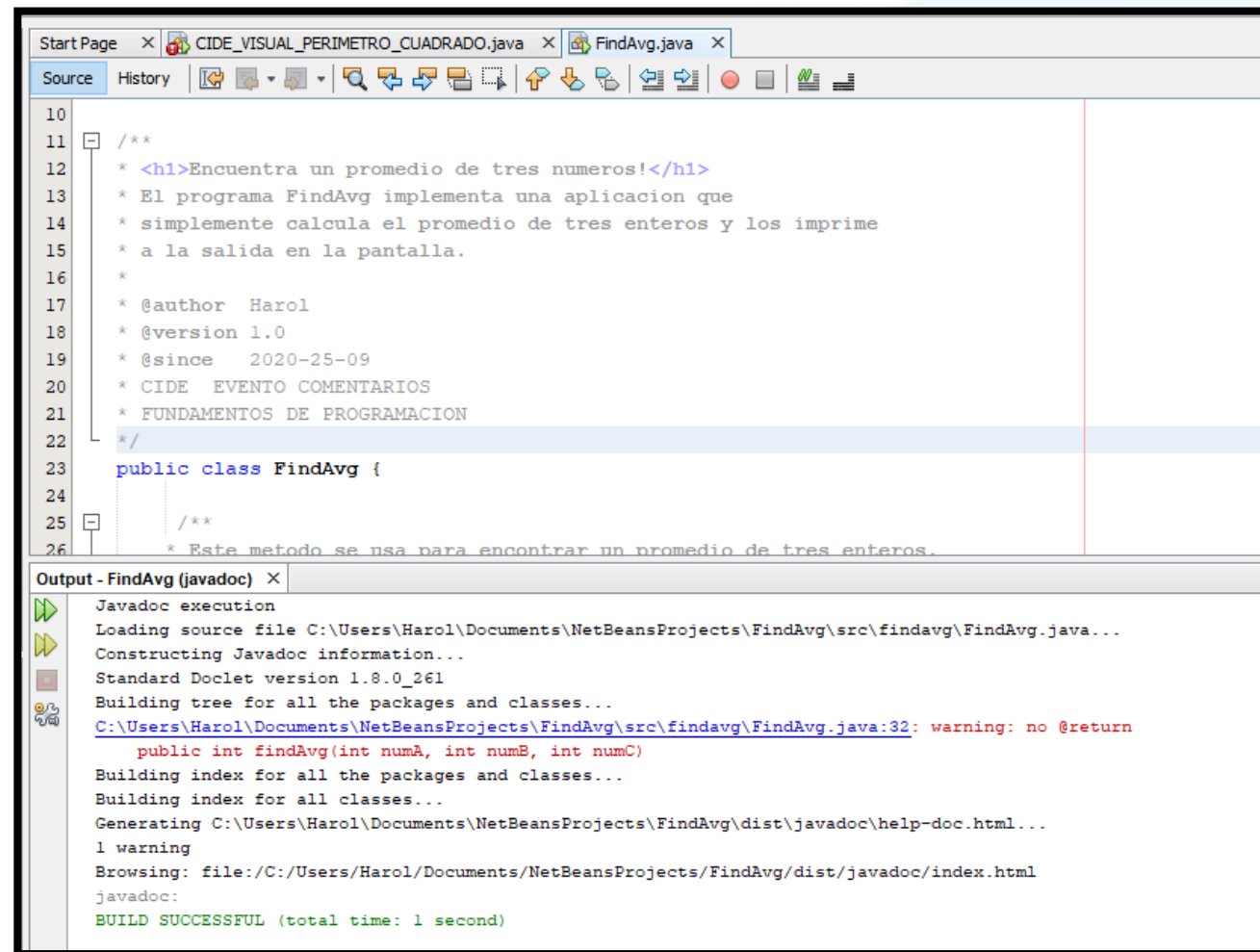
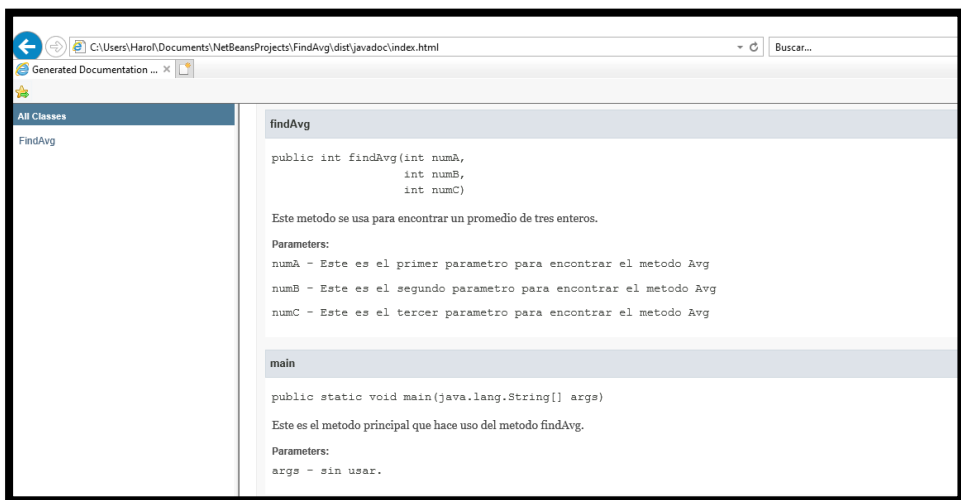
<code>{@linkplain}</code>	Idéntico a <code>{@link}</code> , excepto que la etiqueta del enlace se muestra en texto plano que en la fuente del código.	<code>{@linkplain package.class#etiqueta-miembro}</code>
<code>@param</code>	Agrega un parámetro con el nombre de parámetro especificado seguido de la descripción especificada en la sección "Parámetros".	<code>@param descripción del nombre-parámetro</code>
<code>@return</code>	Agrega una sección "Returns" con el texto de descripción.	<code>@return descripción</code>
<code>@see</code>	Agrega un encabezado "See Also" ("Ver también") con un enlace o entrada de texto que apunta a la referencia.	<code>@see referencia</code>
<code>@serial</code>	Usado en el comentario del documento para un campo serializable predeterminado.	<code>@serial campo-Descripción incluir excluir</code>
<code>@serialData</code>	Documenta los datos escritos por los métodos <code>writeObject()</code> o <code>writeExternal()</code> .	<code>@serialData descripción-datos</code>
<code>@serialField</code>	Documenta un componente <code>ObjectStreamField</code> .	<code>@serialField campo-nombre campo-tipo campo-descripción</code>

<code>{@linkplain}</code>	Idéntico a <code>{@link}</code> , excepto que la etiqueta del enlace se muestra en texto plano que en la fuente del código.	<code>{@linkplain package.class#etiqueta-miembro}</code>
<code>@param</code>	Agrega un parámetro con el nombre de parámetro especificado seguido de la descripción especificada en la sección "Parámetros".	<code>@param descripción del nombre-parámetro</code>
<code>@return</code>	Agrega una sección "Returns" con el texto de descripción.	<code>@return descripción</code>
<code>@see</code>	Agrega un encabezado "See Also" ("Ver también") con un enlace o entrada de texto que apunta a la referencia.	<code>@see referencia</code>
<code>@serial</code>	Usado en el comentario del documento para un campo serializable predeterminado.	<code>@serial campo-Descripción incluir excluir</code>
<code>@serialData</code>	Documenta los datos escritos por los métodos <code>writeObject()</code> o <code>writeExternal()</code> .	<code>@serialData descripción-datos</code>
<code>@serialField</code>	Documenta un componente <code>ObjectStreamField</code> .	<code>@serialField campo-nombre campo-tipo campo-descripción</code>

COMENTARIOS EN LA DOCUMENTACIÓN

@since	Indica a partir de que versión de la API fue incluida la clase o método.	@since release
@throws	Las etiquetas @throws y @exception son sinónimos.	@throws descripción del nombre de la clase
{@value}	Cuando {@value} se usa en el comentario del doc de un campo estático, muestra el valor de esa constante.	{@value package.class#campo}
@version	Agrega un subtítulo de "Versión" con el texto de versión especificado a los documentos generados cuando se usa la opción -version.	@version texto-version

COMENTARIOS EN LA DOCUMENTACIÓN



BUCLAS WHILE JAVA



Un bucle while es una sentencia de control de flujo que permite que el código se ejecute repetidamente en función de una condición booleana dada. El bucle while se puede considerar como una instrucción if repetitiva.

```
void compileFile(final SyntaxNode n) throws CodeException {
    for (Iterator<SyntaxNode> ite = n.getChildren().iterator(); ite.hasNext(); ite.next()) {
        final SyntaxNode child = (SyntaxNode) ite.next();
        final Rule rule = child.getRule();
        if (rule.isPackage() || rule.isClass() || rule.isInterface() || rule.isEnum() || rule.isAnnotation() || rule.isModule()) {
            // Handle package, class, interface, enum, annotation, module
        } else if (rule.isImport()) {
            // Handle import
        } else if (rule.isStatic() || rule.isField() || rule.isMethod() || rule.isConstructor() || rule.isBlock()) {
            // Handle static, field, method, constructor, block
        }
    }
}
```

BUCLAS WHILE JAVA

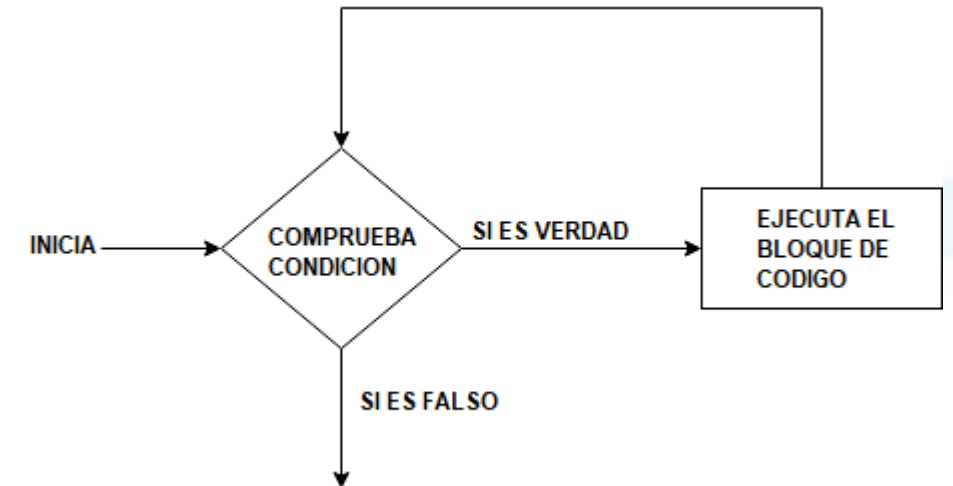


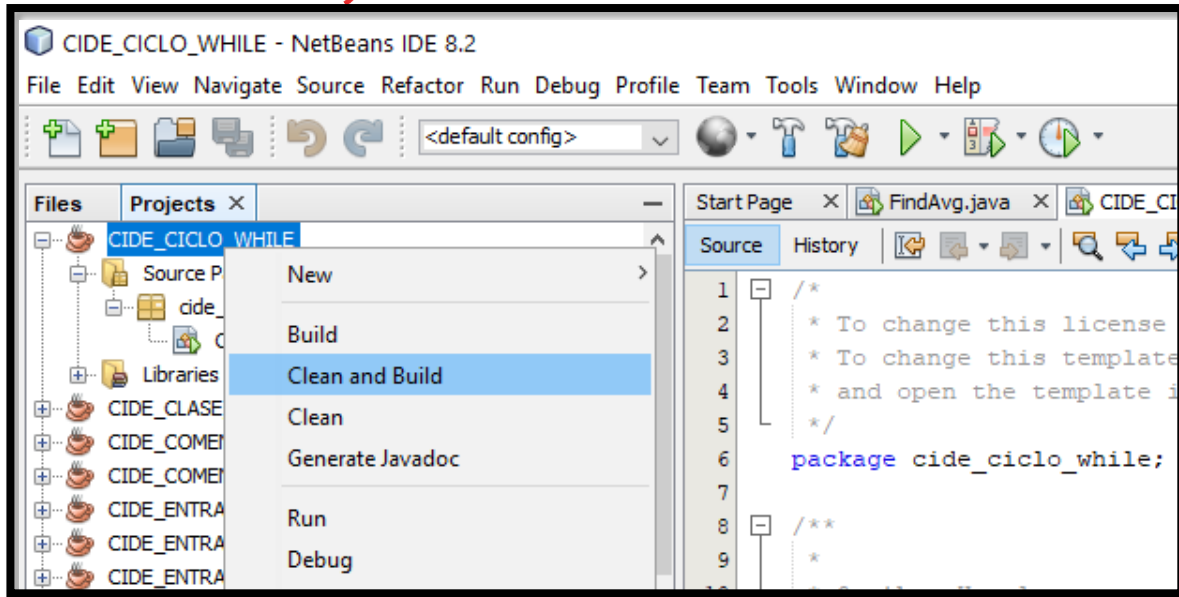
El **WHILE** comienza con la verificación de la condición. Si se evalúa como verdadero, las instrucciones del cuerpo del bucle se ejecutan; de lo contrario, se ejecuta la primera instrucción que le sigue al bucle. Por esta razón, también se llama bucle de **Control de entrada**.

Una vez que la condición se evalúa como verdadera, se ejecutan las instrucciones en el cuerpo del bucle.

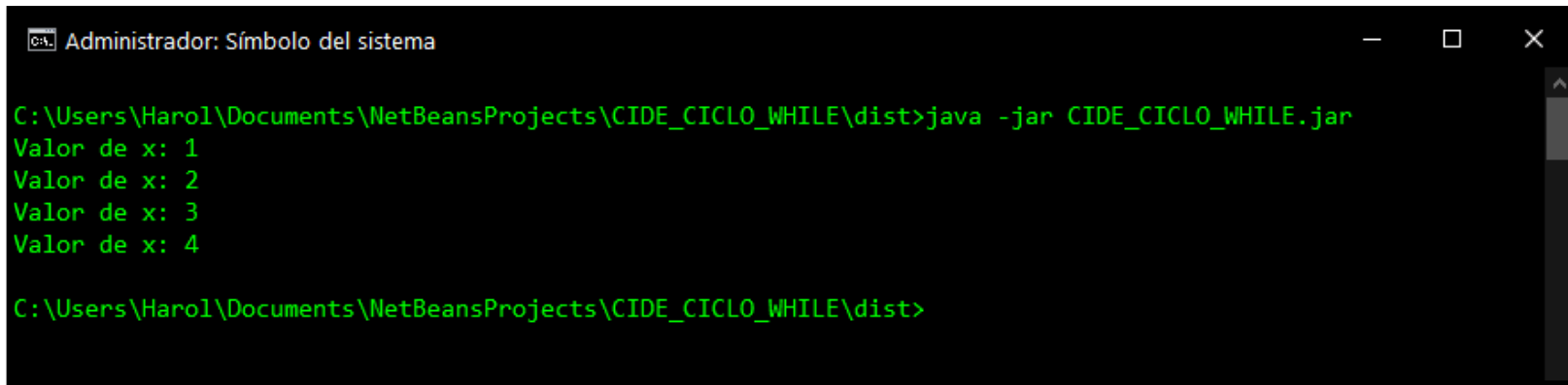
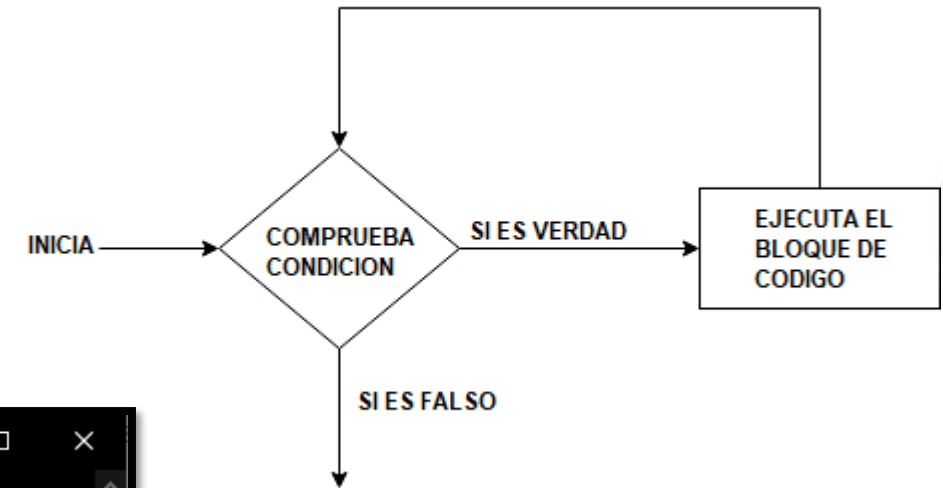
Normalmente, las declaraciones contienen un valor de actualización para la variable que se procesa para la siguiente iteración.

Cuando la condición se vuelve falsa, el ciclo finaliza y marca el final de su ciclo de vida.





BUCLAS WHILE JAVA



```
C:\Users\Harol\Documents\NetBeansProjects\CIDE_CICLO_WHILE\dist>java -jar CIDE_CICLO_WHILE.jar
Valor de x: 1
Valor de x: 2
Valor de x: 3
Valor de x: 4
C:\Users\Harol\Documents\NetBeansProjects\CIDE_CICLO_WHILE\dist>
```

The screenshot shows a Windows command prompt window titled 'Administrador: Símbolo del sistema'. It displays the command to run the Java program and its output, which shows the value of 'x' increasing from 1 to 4.



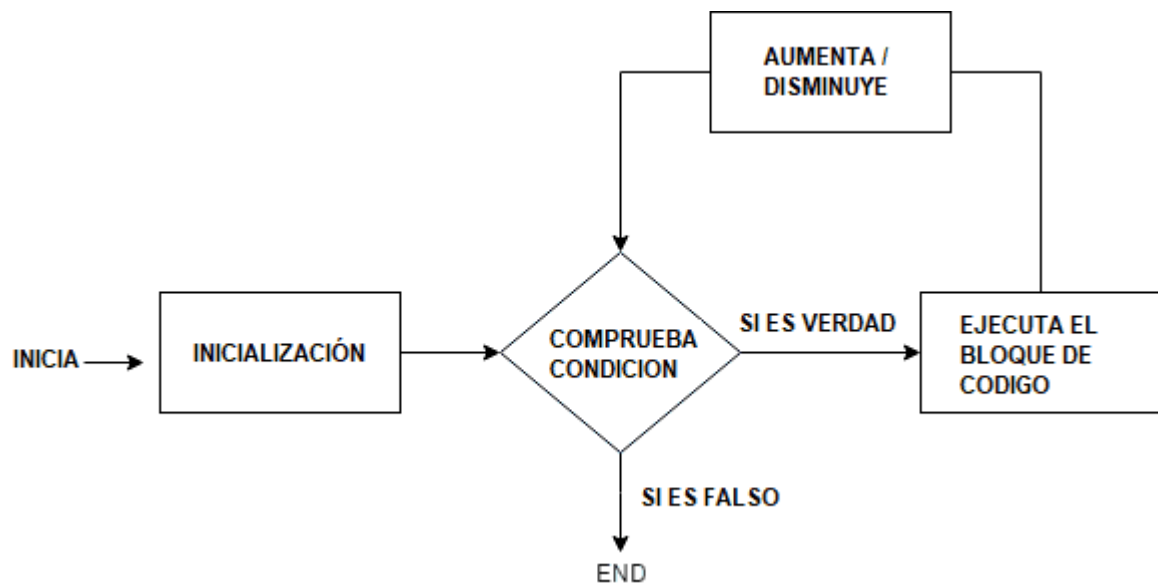
BUCLES FOR JAVA

```
void createFile(final SyntaxNode sn) throws CodeException {  
    for (Iterator<SyntaxNode> ite=sn.getChildren().iterator(); ite.hasNext(); ite.next())  
        createFile((SyntaxNode) ite.next());  
    final Rule rule = cn.getRule();  
    if (rule.getPackageName().equals(rule.getPackageName()) {  
        final Token tok = cn.getTokenByRule(RULE_REFERENCE, rule.getPackageName().chars().toArray());  
    } else if (rule.isImportRule()) {  
        //TODO handle static and final  
        final SyntaxNode sn = cn.getChildByRule(RULE_IMPORT, rule.getPackageName().chars().toArray());  
        final String fullName = ccn.getTokenChars(sn);  
        final String[] parts = fullName.split("\\.");  
    }  
}
```



El bucle FOR proporciona una forma concisa de escribir la estructura de bucle. A diferencia de un ciclo WHILE, una sentencia FOR consume la inicialización, la condición y el incremento/decremento en una línea, proporcionando así una estructura de bucle más corta y fácil de depurar.

BUCLAS FOR JAVA



condición de inicialización: Aquí, inicializamos la variable en uso. Marca el inicio de un ciclo For. Se puede usar una variable ya declarada o se puede declarar una variable, solo local para el bucle.



Condición de prueba: se usa para probar la condición de salida de un bucle. Debe devolver un valor booleano. También es un bucle de control de entrada cuando se verifica la condición antes de la ejecución de las instrucciones de bucle.

Ejecución de instrucción: una vez que la condición se evalúa como verdadera, se ejecutan las instrucciones en el cuerpo del bucle.

Incremento/Decremento: se usa para actualizar la variable para la siguiente iteración.

Terminación de bucle: cuando la condición se vuelve falsa, el bucle termina marcando el final de su ciclo de vida.



BUCLES FOR JAVA

```
*  
* @author Harol  
*/  
// CIDE CICLO FOR  
// FUNDAMENTOS DE PROGRAMACION  
public class CIDE_CICLO_FOR {  
  
    /**  
     * @param args the command line arguments  
     */  
    public static void main(String[] args) {  
        // TODO code application logic here  
        // bucle for comienza cuando x=2  
        // y corre hasta x <=4  
        for (int x = 2; x <= 4; x++)  
            System.out.println("Valor de x: " + x);  
    }  
}
```

```
C:\Users\Harol\Documents\NetBeansProjects\CIDE_CICLO_FOR\dist>java -jar CIDE_CICLO_FOR.jar  
Valor de x: 2  
Valor de x: 3  
Valor de x: 4  
  
C:\Users\Harol\Documents\NetBeansProjects\CIDE_CICLO_FOR\dist>
```

BUCLAS FOR JAVA

```
Output - CIDE_CICLO_FOR (clean,jar) X
ant -f C:\Users\Harol\Documents\NetBeansProjects\CIDE_CICLO_FOR -Dnb.internal.action.name=rebuild clean jar
init:
deps-clean:
Updating property file: C:\Users\Harol\Documents\NetBeansProjects\CIDE_CICLO_FOR\build\build-clean.properties
Deleting directory C:\Users\Harol\Documents\NetBeansProjects\CIDE_CICLO_FOR\build
clean:
init:
deps-jar:
Created dir: C:\Users\Harol\Documents\NetBeansProjects\CIDE_CICLO_FOR\build
Updating property file: C:\Users\Harol\Documents\NetBeansProjects\CIDE_CICLO_FOR\build\build-jar.properties
Created dir: C:\Users\Harol\Documents\NetBeansProjects\CIDE_CICLO_FOR\build\classes
Created dir: C:\Users\Harol\Documents\NetBeansProjects\CIDE_CICLO_FOR\build\empty
Created dir: C:\Users\Harol\Documents\NetBeansProjects\CIDE_CICLO_FOR\build\generated-sources\ap-source-output
Compiling 1 source file to C:\Users\Harol\Documents\NetBeansProjects\CIDE_CICLO_FOR\build\classes
compile:
Created dir: C:\Users\Harol\Documents\NetBeansProjects\CIDE_CICLO_FOR\dist
Copying 1 file to C:\Users\Harol\Documents\NetBeansProjects\CIDE_CICLO_FOR\build
Nothing to copy.
Building jar: C:\Users\Harol\Documents\NetBeansProjects\CIDE_CICLO_FOR\dist\CIDE_CICLO_FOR.jar
To run this application from the command line without Ant, try:
java -jar "C:\Users\Harol\Documents\NetBeansProjects\CIDE_CICLO_FOR\dist\CIDE_CICLO_FOR.jar"
jar:
BUILD SUCCESSFUL (total time: 0 seconds)
```



BUCLAS FOR EACH JAVA

```
void createFile(final SyntaxNode sn) throws CodeException {  
    for (Iterator<SyntaxNode> ite=sn.getChildren().iterator(); ite.hasNext(); )  
        final SyntaxNode child = (SyntaxNode) ite.next();  
        final Rule rule = child.getRule();  
        if (rule.isPackage() || rule.isImport()) {  
            final String pack = child.getCharsByRule(RULE_PACKAGE).getTokensChars();  
            if (rule.isImport()) {  
                //TODO handle static and local  
            }  
            final SyntaxNode n = child.getChildByRule(RULE_IMPORT);  
            final String fullName = n.getTokensChars();  
            final String[] parts = fullName.split("\\.");  
        }  
    }  
}
```



Java también incluye otra versión del bucle for introducido en Java 5. La mejora del bucle for proporciona una forma más sencilla de iterar a través de los elementos de una colección o matriz. Es inflexible y debe usarse solo cuando existe la necesidad de recorrer los elementos de forma secuencial sin conocer el índice del elemento procesado actualmente.



BUCLAS FOR EACH JAVA

```
for (Elemento T:Colección de obj/array)
{
    declaraciones
}
```

```
C:\Users\Harol\Documents\NetBeansProjects\CIDE_CICLO_FOR_EACH\dist>java -jar CIDE_CICLO_FOR_EACH.jar
Harol
Torres
CIDE
C:\Users\Harol\Documents\NetBeansProjects\CIDE_CICLO_FOR_EACH\dist>
```

```

*
* @author Harol
*/
// CIDE CICLO FOR EACH
// FUNDAMENTOS DE PROGRAMACION
public class CIDE_CICLO_FOR_EACH {

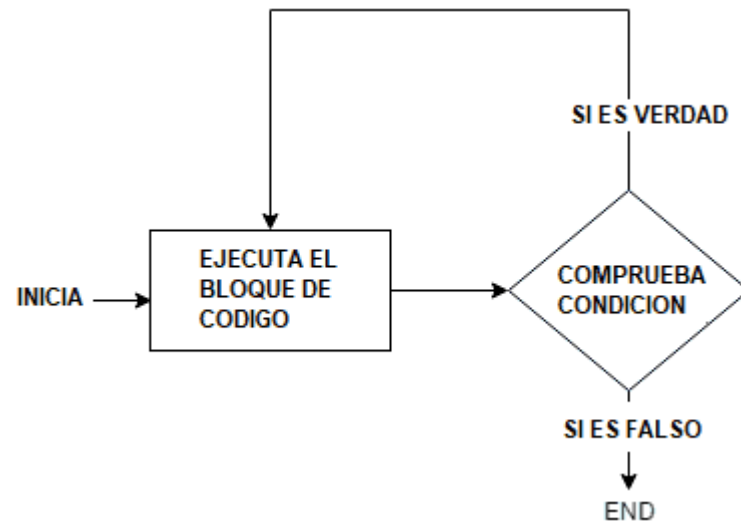
    /**
     * @param args the command line arguments
     */
    public static void main(String[] args) {
        // TODO code application logic here
        String array[] = {"Harol", "Torres", "CIDE"};

        //mejorado para for
        for (String x:array)
        {
            System.out.println(x);
        }

        /*bucle for para la misma función
        for (int i = 0; i < array.length; i++)
        {
            System.out.println(array[i]);
        }
        */
    }
}
```



BUCLES DO WHILE JAVA



El bucle do while es similar al while con la única diferencia de que comprueba la condición después de ejecutar las instrucciones, y por lo tanto es un ejemplo de Exit Control Loop (Salir del bloque de control).

BUCLES DO WHILE JAVA

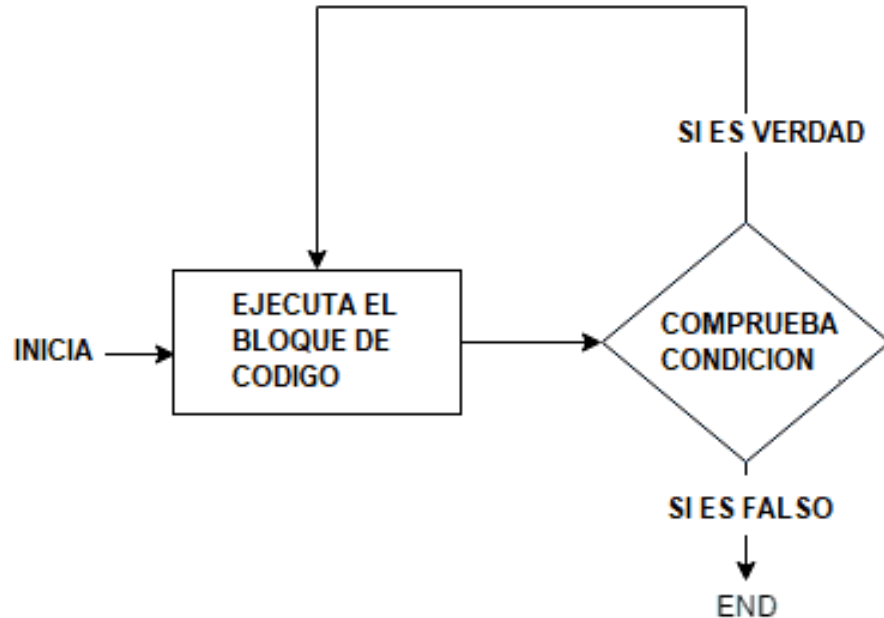


El bucle do while comienza con la ejecución de la(s) declaración(es). No hay verificación de ninguna condición la primera vez.

Después de la ejecución de los enunciados, y la actualización del valor de la variable, la condición se verifica para el valor verdadero o falso. Si se evalúa como verdadero, comienza la siguiente iteración del ciclo.

Cuando la condición se vuelve falsa, el ciclo finaliza y marca el final de su ciclo de vida.

Es importante tener en cuenta que el bucle do-while ejecutará sus declaraciones al menos una vez antes de que se verifique cualquier condición, y por lo tanto es un ejemplo de bucle de control de salida.



BUCLES DO WHILE JAVA

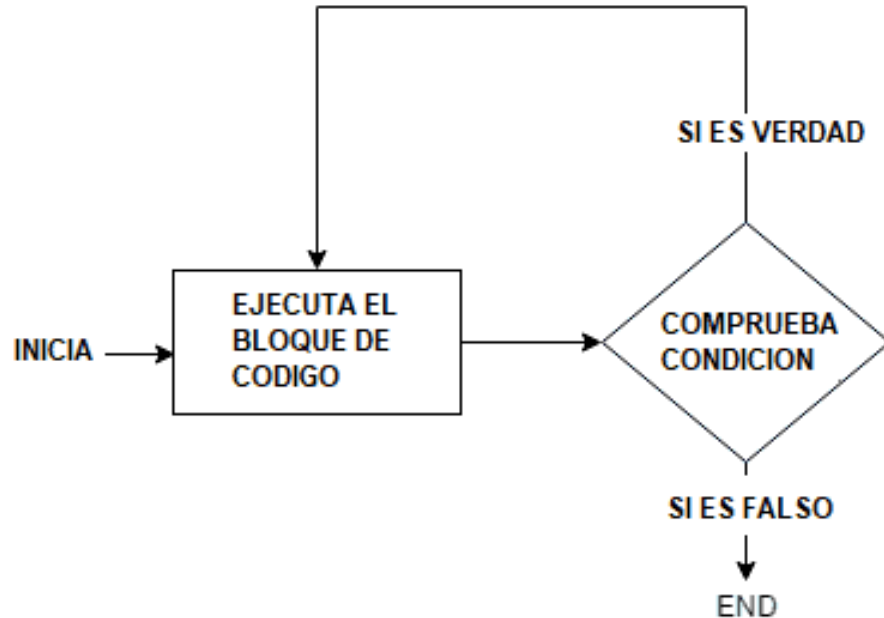


El bucle do while comienza con la ejecución de la(s) declaración(es). No hay verificación de ninguna condición la primera vez.

Después de la ejecución de los enunciados, y la actualización del valor de la variable, la condición se verifica para el valor verdadero o falso. Si se evalúa como verdadero, comienza la siguiente iteración del ciclo.

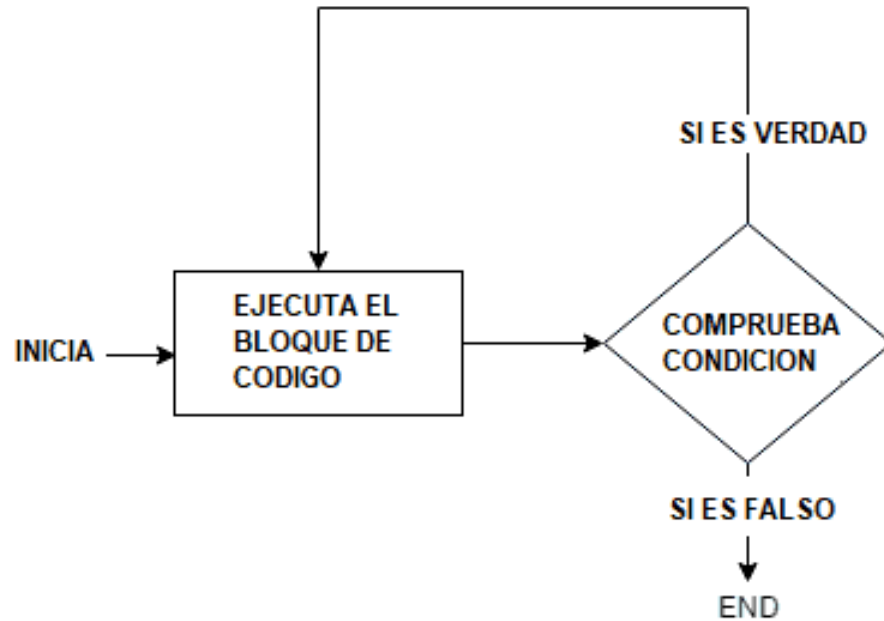
Cuando la condición se vuelve falsa, el ciclo finaliza y marca el final de su ciclo de vida.

Es importante tener en cuenta que el bucle do-while ejecutará sus declaraciones al menos una vez antes de que se verifique cualquier condición, y por lo tanto es un ejemplo de bucle de control de salida.





BUCLAS DO WHILE JAVA



```
public static void main(String[] args) {  
    // TODO code application logic here  
    int x = 21;  
    do  
    {  
        //El código dentro del do se imprime incluso  
        //si la condición es falsa  
        System.out.println("Valor de x :" + x);  
        x++;  
    }  
    while (x < 20);  
}
```

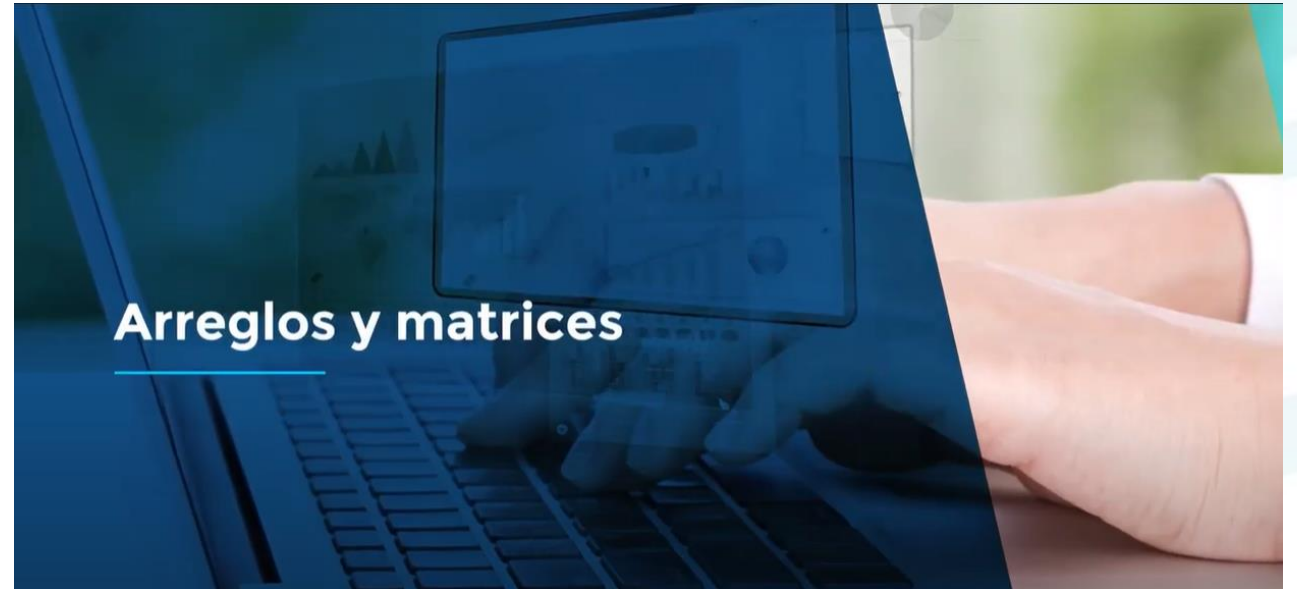
IDE breadcrumb: cide_ciclo_do_while.CIDE_CICLO_DO_WHILE > main > do ... while (x < 20) >

Output window: out - CIDE_CICLO_DO_WHILE (run) X

run:
Valor de x :21
BUILD SUCCESSFUL (total time: 0 seconds)

```
C:\Users\Harol\Documents\NetBeansProjects\CIDE_CICLO_DO_WHILE\dist>java -jar CIDE_CICLO_DO_WHILE.jar  
Valor de x :21  
  
C:\Users\Harol\Documents\NetBeansProjects\CIDE_CICLO_DO_WHILE\dist>
```

ARREGLOS Y MATRICES



<https://www.youtube.com/watch?v=W8QwbhynEmc&t=267s>



Politécnico
Internacional



https://www.youtube.com/watch?v=mEvjuG_tbqY

Tutorial Java - 9. Arreglos

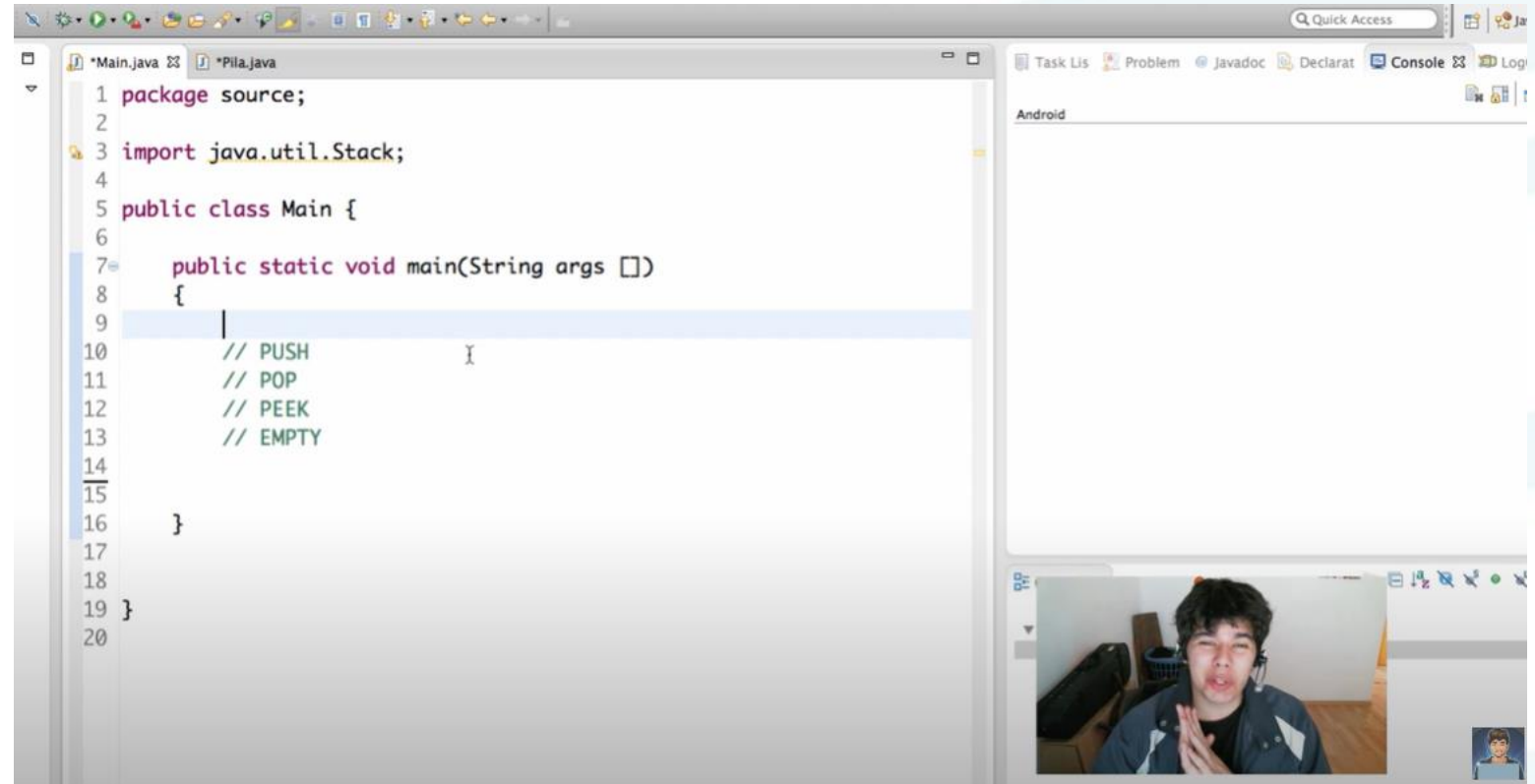


Politécnico
Internacional



Capítulo 6. Listas Enlazadas URL

https://www.youtube.com/watch?v=r_ttiJ45mfw



```
1 package source;
2
3 import java.util.Stack;
4
5 public class Main {
6
7     public static void main(String args [])
8     {
9         |
10        // PUSH
11        // POP
12        // PEEK
13        // EMPTY
14
15
16    }
17
18 }
19
20
```

Estructura de datos en Java

<https://www.youtube.com/watch?v=BdgdjTFpC6g>



Arreglos bidimensionales (Matrices dinámicas)

<https://www.youtube.com/watch?v=3FIfrC1kG0>

AVANCES **INNOVA** COLOMBIA

¿Preguntas?

RETO 1 + 2

CONCLUSIONES